

task-03-find-the-mistakes

Page 1 : name

Page2 : the mistakes that was in the code and the right use

Page3: the correct use in code and why it's important for mistake number 1,2 and 3

Page4: the correct use in code and why it's important for mistake number 4,5 and 6

In fixing the **Churn_Find_The_Mistakes** task i read the code and the data to Track all details miss using sum libraries that was in the code and haven't used or not useful with the project This is a list that have all the miss uses in the code or that haven't used in the right way

- **Missing Value Imputation:**

- *Wrong:* Used `mean()` to fill missing `TotalCharges`, which is heavily skewed by extreme values (outliers).
- *Fix:* Switched to `median()`, which is robust to outliers.

- **Categorical Encoding:**

- *Wrong:* Used `LabelEncoder()` for all categorical features, which falsely tells the model that the categories have a numbered, ranked order (e.g., `Apple=1`, `Banana=2`).
- *Fix:* Mapped the target directly (1/0) and used `pd.get_dummies(drop_first=True)` for One-Hot Encoding, which properly handles unordered categories while avoiding the dummy variable trap.

- **Data Leakage during Scaling:**

- *Wrong:* Applied `StandardScaler` to the entire dataset *before* splitting it, allowing the training data to "peek" at the distribution of the test data.
- *Fix:* Split the data first. Then applied `.fit_transform()` exclusively to `X_train`, and only applied `.transform()` to `X_test`. Also added `stratify=y` to the split to maintain the churn ratio.

- **Data Leakage during Hyperparameter Tuning:**

- *Wrong:* Used a manual for loop to test different values of `C` directly against the test set (`X_test`, `y_test`). This essentially uses the test set to train the model, ruining its ability to measure real-world performance.
- *Fix:* Implemented `GridSearchCV` to find the best `C` using 5-fold cross-validation (`cv=5`) strictly on the training set.

- **Evaluating on the Wrong Dataset:**

- *Wrong:* Evaluated the final model's predictions on `X_train` against `y_train`, testing the model on the exact same data it just memorized.
- *Fix:* Evaluated the final, optimized model on the unseen `X_test_scaled` data.

- **Misleading Evaluation Metric:**

- *Wrong:* Relied purely on `accuracy_score` based on the false assumption in the notes that the classes were balanced (Churn datasets are highly imbalanced).
- *Fix:* Switched the `GridSearch` scoring metric to `f1` and used `classification_report` at the end to view Precision, Recall, and F1-scores for a true picture of performance.

1. Missing Value Imputation

Why it matters: The mean is easily distorted by extreme outliers (e.g., a few customers with massive total charges). The median simply picks the exact middle value, providing a much more realistic baseline for missing data without warping your dataset.

- **Old Code:** `df['TotalCharges'].fillna(df['TotalCharges'].mean())`
- **New Code:** `df['TotalCharges'].fillna(df['TotalCharges'].median())`

2. Categorical Encoding

Why it matters: `LabelEncoder` turns categories into sequential numbers (e.g., DSL=1, Fiber=2, No Internet=3). Machine learning models interpret this as a mathematical ranking, assuming "3 is greater than 1," which is mathematically false for categories. `get_dummies` creates separate Yes/No columns for each category (One-Hot Encoding), preventing the model from learning false relationships.

- **Old Code:**
for col in df.select_dtypes(include='object').columns:
 df[col] = encoder.fit_transform(df[col])
- **New Code:**
df = pd.get_dummies(df, drop_first=True)

3. Data Leakage During Scaling

Why it matters: In the real world, you cannot scale your data based on future customers you haven't seen yet. If you scale the whole dataset *before* splitting, information about the test set's distribution "leaks" into the training set. You must split first, let the scaler learn the rules (`fit`) only from the training set, and then apply those rules (`transform`) to the test set.

Old Code:

```
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y...)
```

New Code:

```
X_train, X_test, y_train, y_test = train_test_split(X, y...)
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

4. Data Leakage During Hyperparameter Tuning

Why it matters: The old code used the `X_test` data over and over again to check which value of `C` was best. By doing this, you are explicitly optimizing the model to memorize the test set, ruining its purpose as a final, unbiased "unseen data" check. `GridSearchCV` uses cross-validation internally on the *training* data only, keeping the test data completely pure until the very end.

- **Old Code:**

```
for c in [0.001, 0.01, 0.1, 1, 10, 100]:
```

```
    m = LogisticRegression(C=c).fit(X_train, y_train)
```

```
    acc = accuracy_score(y_test, m.predict(X_test)) # Using test data to tune!
```

- **New Code:**

```
grid = GridSearchCV(LogisticRegression(), param_grid, cv=5)
```

```
grid.fit(X_train_scaled, y_train) # Tuned entirely on training data
```

5. Evaluating on the Wrong Dataset

Why it matters: Testing a model on `X_train` is like giving a student an exam where they already saw the answer key. It will give you a falsely high score because the model has already memorized that exact data. You must always predict on the test set to see if the model actually learned the underlying patterns.

- **Old Code:** `preds = model.predict(X_train)`

- **New Code:** `preds = best_model.predict(X_test_scaled)`

6. Misleading Evaluation Metric

Why it matters: Customer churn datasets are inherently imbalanced (e.g., 80% stay, 20% leave). If a broken model simply predicts "Stay" for every single customer, it will achieve 80% accuracy while failing to identify a single churner. Accuracy is a useless metric here. The F1-score and classification report look at Precision and Recall, proving whether the model can actually catch the minority class (the churners).

- **Old Code:** `accuracy_score(y_train, preds)`

- **New Code:**

```
d = GridSearchCV(..., scoring='f1') # Optimizing for F1 instead of accuracy
```

```
print(classification_report(y_test, preds)) # Shows precision, recall, and f1
```